Neural Computing and Applications (2022) 34:21793–21808
https://doi.org/10.1007/s00521-022-07620-8

ORIGINAL ARTICLE

A novel rough set-based approach for minimum vertex cover of hypergraphs

Qian Zhou[1] · Xiaojun Xie[2] · Hua Dai[3] · Weizhi Meng[4]

Received: 27 April 2022 / Accepted: 4 July 2022 / Published online: 3 August 2022

Abstract

Minimum vertex covering has been widely used and studied as a general optimization problem. We focus on one of its variation: minimum vertex cover of hypergraphs. Most existed algorithms are designed for general graphs, where each edge contains at most two vertices. Moreover, among these algorithms, rough set-based algorithms have been proposed recently and attract many researchers sight. However, they are not efficient enough when the number of nodes and hyperedges scale largely. To address these limitations, we propose a novel rough set-based approach by combining rough set theory with the stochastic local search algorithm. In this approach, three improvements have been introduced, i.e., (1) fast relative reduct construction method, which can quickly achieve a relative reduct, and it is based on low-complexity heuristics; (2) (p, q)-reverse incremental verification mechanism, which uses incremental positive region update technology to quickly verify whether a required attribute pair can be found; (3) adjusting iterative process rules, the main purposes of these rules are avoiding repeated computation and jumping out of local optimum. Finally, by comparing groups of benchmark graphs and hypergraphs with the existing algorithms based on rough sets, experimental results presents the advantages and limitations of our proposed approach.

Keywords Minimum vertex cover · Hypergraph · Rough set · Attribute reduction

1 Introduction

A typical NP-hard combinatorial problem, termed as minimum vertex cover problem (MVCP), is to finding a minimum subset of vertices, which includes at least one endpoint of each edge. Its decision version, the vertex cover problem, was one of Karp's twenty one NP-complete problems presented in his 1972 seminal paper [1]. Since then, MVCP has been used a few dozen times in a very diverse areas of applications, let us mention only [2, 3, 5–7, 9]. From the above list, for example [5] deals with communications problem in wireless sensor networks while [6] with phase measurement units placement in power system. In addition, under time constraints in the Internet of vehicles, the problem in solving the data broadcast problem can be described as the minimum vertex cover problem of bipartite graph.

As the problem is NP-hard, finding exact solutions is usually not feasible for bigger graphs and heuristic algorithms are one of the possible solutions. Among many types of heuristic algorithms, the local search-based algorithms often provide a good performance for a wide variety

Xiaojun Xie, Hua Dai and Weizhi Meng contributed equally to this work.

✉ Qian Zhou
zhouqian@njupt.edu.cn

Xiaojun Xie
xxj@njau.edu.cn

Hua Dai
daihua@njupt.edu.cn

Weizhi Meng
weme@dtu.dk

[1] School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing 210003, Jiangsu, People's Republic of China

[2] College of Artificial Intelligence, Nanjing Agricultural University, Nanjing 210095, Jiangsu, People's Republic of China

[3] School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, Jiangsu, People's Republic of China

[4] Department of Applied Mathematics and Computer Science, Technical University of Denmark, Lyngby 2800 Kgs, Copenhagen, Denmark

of cases and many of them have been provided in last two decades [12–14, 16].

Hypergraphs whose hyperedges own an arbitrary number of vertices, are generalizations of graphs [19, 23]. While MVCP can be extended to hypergraphs in a rather natural way [23], efficient local search-based algorithms designed for graphs usually do not work well for hypergraphs, so alternative methods have to be considered.

Rough set theory, as a theoretical base of various algorithms, has been quiet popular used in pattern recognition, data mining, machine learning and many other areas [24–26]. Recently, some interesting connections between rough sets and hypergraphs have been discovered and analyzed [27–30]. It turns out that some hypergraphs problems can be transformed into equivalent rough sets problems and vice versa. In particular, MVCP in hypergraphs corresponds closely to the attribute reduction problem in rough set theory, which is also NP-hard [31, 32], and also has many efficient heuristic solutions [33–35] that can be used in real-world application.

Recently, the rough sets-based attribute reduction techniques become a useful tool for solving MVCP for hypergraphs. The rough set based method makes it easier to handle MVCP of a hypergraph than the classical local search-based approach. But the results obtained by rough set-based approach are usually far from the optimal solution.

Considering that the existing local search approaches cannot deal with hypergraph, and the rough set-based approaches are not efficient enough when the number of nodes and hyperedges scale largely. To overcome these limitations, we propose a new approach where we combine rough set theory with stochastic local search approaches. From a given hypergraph, we provide a method to derive an appropriate rough sets decision table and the calculation formulas for computing positive regions and core attributes. Then, we design and apply a stochastic local search algorithm for minimal attribute reduction.

In summary, the contributions of this paper are threefold:

1. a novel efficient algorithm, based on rough sets and stochastic local search is proposed for solving MVCP for a given hypergraph;
2. three improvements that enhance the efficiency of our proposed approach, namely: fast relative reduct construction method, (p, q)-reverse incremental verification mechanism and appropriate adjustment strategies, are designed and implemented;
3. groups of comparative experiments are employed to verify the efficiency and effectiveness of this proposed algorithm compared with well-known benchmarks.

The rest of the paper is organized as follows. Section 2 summarizes some related work for MVCP. In Sect. 3, some basic concepts from rough set theory and graph theory are recalled. Section 4 discusses a derivation of the induced decision table from a given hypergraph. In Sect. 5, we show how rough sets minimal attribute reduction algorithm can be used for solving MVCP of a given hypergraph. Section 6 illustrates a number of experimental results on well-known benchmarks. Section 7 discusses the conclusions and further research.

2 Related work

Within a factor of 1.3606, MVCP is NP-hard to approximate [20]. Many scholars have devoted themselves to proposing efficient methods in the past two decades. The existing approaches for MVCP can be divided into two types: exact algorithms and approximate algorithms. In the exact algorithms, the most common method is branching algorithms, such as LP-based branch-and-cut and branch-and-bound [21, 22]. The exact algorithm usually has high time complexity, although it can find optimal solution with enough waiting time. It could have the optimal result immediately for the small-scale instances. However, for large-scale instances, it costs a long waiting time or could not make it. Due to the increasing scale of data sets, more and more scholars prefer to study the approximation algorithm. The existing approximate algorithms mainly focus on three types: evolutionary-based algorithms, local search-based algorithms and rough set-based algorithms. Table 1 presents a summary of few of the approximate algorithms.

The memory-based best response rule in MBR is synchronous updating for the spatial snowdrift game with memory, whose strict Nash equilibrium are vertex cover states of the network. GMA-MVC for the MVCP always obtains a better solution than MBR, but the game evolution is easy to be trapped in local optima.

Strictly speaking, COVER was not designed for MVCP, but it provides an innovative method that have often been used later. Inspired by COVER, the key point of EWLS is to find a vertex set that provides a tight upper bound on the size of the minimum vertex cover. The other six local search-based algorithms design some improvement strategies based on EWLS to improve the optimization ability of the algorithm. For example, EWCC is more efficient than EWLS and contains a strategy called configuration checking for handling the cycling problem in local search. In NuMVC, two-stage exchange strategy selects two vertices to exchange separately and performs the exchange in two stages, and edge weighting with forgetting strategy not only increases weights of uncovered edges, but also

Table 1 Approximate algorithms for MVCP

Types	Name	Research objects	Descriptions
Evolutionary	MBR [8]	Networks	A snowdrift game framework for vertex cover of agent-based networks is constructed as a new approach from the perspective of evolutionary game theory
	GMA-MVC [9]	Networks	An optimization framework to vertex cover of networks based on spatial snowdrift game
Local search	COVER [10]	General graphs	A stochastic local search algorithm for k-vertex cover problem and uses edge weights to guide the local search
	EWLS [11]	General graphs	An edge weighting local search for MVCP based on the idea of extending a partial vertex cover into a vertex cover
	EWCC [12]	General graphs	It is designed based on EWLS and has no instance-dependent parameters
	NuMVC [13]	General graphs	It contains two new strategies in local search: two-stage exchange and edge weighting with forgetting
	TwMVC [14]	General graphs	A two weighting local search algorithm for MVCP, i.e., edge weighting and vertex weighting
	FastVC [15]	Massive graphs	In massive sparse graphs, a small vertex cover to be found
	LSTP [16]	General graphs	An efficient local search algorithm to solve the generalized vertex cover problem with two improvement: tabu strategy and perturbation mechanism
	EAVC [17]	Massive graphs	A local search algorithm that employs a new edge weighting method is termed as edge age based best from multiple selections (EABMS)
Rough set	VCRS [30]	General graphs	A heuristic attribute reduction algorithm based on Shannon's information entropy
	VCAR [36]	General graphs	An accelerating algorithm for the MVCP based on the positive region
	IFS-S [37]	Dynamic hypergraphs	A dimension incremental attribute reduction algorithm for MVCP with a single vertex arriving
	IFS-M [37]	Dynamic hypergraphs	A dimension incremental attribute reduction algorithm for MVCP with multiple vertices arriving
	SPS-G [39]	General graphs	A novel algorithm based on minimal elements of discernibility matrix
	IH-R [40]	Weighted graphs	An improved heuristic algorithm for the weighted version of MVCP based on test-cost-sensitive rough set
	IQPSO-R [40]	Weighted graphs	An immune quantum-behaved particle swarm and test-cost-sensitive rough set-based algorithm for the weighted version of MVCP

decreases some weights for each edge periodically. TwMVC designs a vertex weighting scheme to address the shortcoming of current local search algorithms using edge weighting techniques, and combines this scheme within NuMVC. FastVC is proposed for solving MVCP in very large scale real-world graphs based on two new heuristics. One heuristic strikes a good balance between solution quality and time complexity, and the other one approximates the minimum loss heuristic very well and lowers the complexity of each removing-vertex selection step from $O(|V|)$ to $O(1)$. These two heuristics are the fundamental reason why the performance of FastVC is better than that of NuMVC on massive graphs. In LSTP, tabu strategy can prevent the local search from immediately returning to a previously visited candidate solution and avoiding the cycling problem, and perturbation mechanism can help the search to escape from deep local optima and to bring diversification into the search. In EAVC, EABMS inherits an $O(1)$ time-complexity from best from multiple selections (BMS), and effectively combines BMS with an edge-aging technique.

For the existing rough set-based algorithms, VCRS first converts the vertex cover problem of graphs into the attribute reduction problem of information system, the biggest drawback of VCRS is that its time complexity is too high. To improve VCRS, VCAR transforms the vertex cover problem of graph into the attribute reduction problem of decision table, and contains a fast algorithm for

computing the positive region. Compared with VCRS, VCAR improves significantly the running time of the algorithm. IFS-S and IFS-M study MVCP from the perspective of incremental attribute reduction with single vertex arriving and multiple vertices arriving, respectively. IFS-S can obtain a better solution than VCAR but it sacrifices the computational time, and IFS-M is faster than VCAR but the result is not as good as VCAR. However, IFS-S and IFS-M are suitable for dynamic hypergraph environments. SPS-G employs an approach of discernibility matrix in rough sets to deal with MVCP, but SPS-G does not seems to have any obvious advantage over VCAR.

IH-R is actually a traditional heuristic algorithm. The limitations of IH-R are as follows: (1) the results of IH-R depend on the setting of λ; (2) it needs to run many times to achieve the best results. IQPSO-R is an swarm intelligence optimization algorithm, and its performance on small size of instances still has some advantages but IQPSO-R usually fails to find the optimal solutions on large instances. Comparing these three types of algorithms, local search algorithms are superior to evolutionary-based algorithms and rough set-based algorithms, and rough set-based algorithms is one of the known effective methods to solve MVCP of hypergraphs. To design an effective minimum vertex cover algorithm for hypergraph, this paper combines the local search algorithm and rough sets to explore MVCP of hypergraph.

3 Preliminaries

This section recalls some basic concepts and definitions from graph theory and rough sets [19, 24].

A hypergraph is a generalized form of a graph, where each edge may contain any number of vertices. It can be represented as $G = (V, E)$, where $V = \{v_1, v_2, \cdots, v_n\}$ is the set of vertices and $E = \{e_1, e_2, \cdots, e_m\} \subseteq 2^V$ is the set of hyperedges. Figure 1 provides an example of a hypergraph with $V = \{v_1, v_2, \cdots, v_6\}$,$E = \{e_1, e_2, \cdots, e_6\}$ and $e_1 = \{v_1, v_3, v_4, v_6\}$,$e_2 = \{v_1, v_4, v_5, v_6\}e_3 = \{v_1, v_3, v_4, v_5\}e_4 = \{v_2, v_4\}$ $e_5 = \{v_2, v_3\}e_6 = \{v_2, v_6\}$.

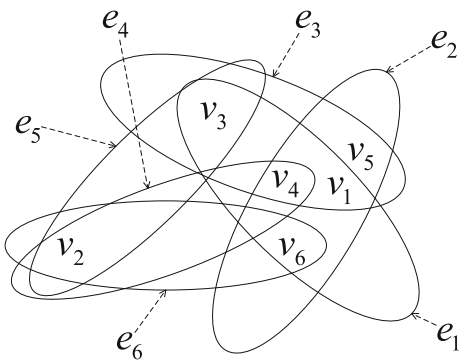


Fig. 1 An example of a simple

Definition 1 Let $G = (V, E)$ be a hypergraph and let $K \subseteq V$ [37].

1. A set K is called a ***vertex cover*** if and only if every $e \in E$ has at least one endpoint in K.
2. A set K is called a ***unique vertex cover*** if and only if it is a vertex cover and we have K' is not a vertex cover for each $K' \subsetneq K$.
3. A set K is called a ***minimum vertex cover*** if and only if it is a vertex cover and we have $|K| \leq |K'|$ for any other reduct K'.

Given a hypergraph $G = (V, E)$, we construct a Boolean function as follows:

$$f_G = \wedge\{\vee V(e) \mid \forall e \in E\} \tag{1}$$

where $V(e)$ represents all vertex sets connected by hyperedge e.

Lemma 1 *Let $G = (V, E)$ be a hypergraph and the Boolean function of hypergraph is f_G, a vertex set $K \subseteq V$ is a minimal vertex cover if and only if $\wedge K$ is the prime implicants of the Boolean function f_G* [37].

By using the law of absorption and the law of distribution, the Boolean function f_G transformed from conjunctive normal form to disjunctive normal form, i.e., $f_G = \vee\{\wedge K | \forall K \in \mathsf{Cov}(G)\}$, where K is a minimal vertex cover and $\mathsf{Cov}(G)$ represents a collection of all minimal vertex cover.

Example 1 For a hypergraph $G = (V, E)$ in Fig. 1, the conjunctive normal form of Boolean function f_G is expressed as follows: $f_H = (v_1 \vee v_3 \vee v_4 \vee v_6) \wedge (v_1 \vee v_4 \vee v_5 \vee v_6) \wedge (v_1 \vee v_3 \vee v_4 \vee v_5) \wedge (v_2 \vee v_4) \wedge (v_2 \vee v_3) \wedge (v_2 \vee v_6)$. By using the law of absorption and the law of distribution, we obtain the disjunctive normal form of Boolean function f_G is $f_H = (v_1 \wedge v_2) \vee (v_2 \wedge v_4) \vee (v_2 \wedge v_3 \wedge v_5) \vee (v_2 \wedge v_3 \wedge v_6) \vee (v_2 \wedge v_5 \wedge v_6) \vee (v_3 \wedge v_4 \wedge v_6)$.Therefore, there are six unique vertex covers, namely $K_1 = \{v_1, v_2\}$, $K_2 = \{v_2, v_4\}$, $K_3 = \{v_2, v_3, v_5\}$, $K_4 = \{v_2, v_3, v_6\}$, $K_5 = \{v_2, v_5, v_6\}$ and $K_6 = \{v_3, v_4, v_6\}$, and two minimum vertex covers, i.e., $K_1 = \{v_1, v_2\}$ and $K_2 = \{v_2, v_4\}$.

Throughout this paper, we will assume that *decision tables* representing rough sets information systems *have **only one** decision attribute*, so a simpler notation will be used.

A *decision table* is a triple $S = (U, A \cup \{d\}, f)$, where $U = \{x_1, x_2, \ldots, x_{|U|}\}$ is a finite nonempty set of objects, called *universe* $A = \{a_1, a_2, \ldots, a_{|A|}\}$ is a finite nonempty set of *conditional attributes*, d is the *decision attribute*, and $f : U \times (A \cup \{d\}) \rightarrow \mathcal{V}$ is an *information function*, where

$\mathcal{V}$ is *domain* of attributes $A \cup \{d\}$. Moreover, for every $a \in A \cup \{d\}$, we have $\mathcal{V}_a = \{f(x,a) \mid x \in U\}$.

For any equivalence relation $R \subseteq U \times U$, where U is a set, $[x]_R$ denotes the equivalence class containing $x \in U$, i.e., $[x]_R = \{y \mid (x,y) \in R\}$, and U/R denotes the partition of U defined by R, i.e., $U/R = \{[x]_R \mid x \in U\}$.

Let $S = (U, A \cup \{d\}, f)$ be a decision table. For every nonempty $B \subseteq A$ or $B = \{d\}$, we define $\mathsf{ind}(B)$, *the indiscernibility relation induced by* B, as:

$$\mathsf{ind}(B) = \{(x,y) \mid x,y \in U \wedge \forall a \in B, f(x,a) = f(y,a)\}. \tag{2}$$

The relation $\mathsf{ind}(B)$ is clearly an equivalence relation on U, so each $U/\mathsf{ind}(B)$ is a proper partition of U.

- To simplify notation, for every $U' \subseteq U$, we will often write U'/d instead of more formal $U'/\mathsf{ind}(\{d\})$.

For each pair $(x,y) \in \mathsf{ind}(B)$, if $f(x,d) \neq f(y,d)$, then (x, y) is called an *inconsistent objects pair*. For each nonempty $B \subseteq A$, we define the set of its inconsistent objects pairs, denoted $\mathsf{iop}(B,d)$, as:

$$\mathsf{iop}(B,d) = \{(x,y) \in \mathsf{ind}(B) \mid f(x,d) \neq f(y,d)\}. \tag{3}$$

For every nonempty $B \subseteq A$, and every $X \subseteq U$, we define $B_-(X)$, the *B-lower approximation* of X, as $B_-(X) = \{x \in U \mid [x]_{\mathsf{ind}(B)} \subseteq X\}$. For every nonempty $B \subseteq A$ and every $U' \subseteq U$, we define the ***positive region*** d over U' with respect to B as:

$$\mathsf{POS}_B^{U'}(d) = \bigcup_{X \in U'/d} B_-(X). \tag{4}$$

When $U' = U$, which is the most popular case, we will just write $\mathsf{POS}_B(d)$ instead of $\mathsf{POS}_B^U(d)$. Note that we always have $\mathsf{POS}_B^{U'}(d) \subseteq U'$. Also, if $(x,y) \in \mathsf{iop}(B,d)$, then $x \notin \mathsf{POS}_B(d)$ and $y \notin \mathsf{POS}_B(d)$.

Definition 2 Let $S = (U, A \cup \{d\}, f)$ be a decision table and let $B \subseteq A$,

1. A set B is called a ***relative attribute reduct*** if and only if $\mathsf{POS}_B(d) = \mathsf{POS}_A(d)$.
2. A set B is called an ***attribute reduct*** if and only if it is a relative reduct and we have $\mathsf{POS}_{B'}(d) \neq \mathsf{POS}_B(d)$ for each $B' \subsetneq B$.
3. A set B is called an ***minimal attribute reduct*** if and only if it is a reduct and we have $|B| \leq |B'|$ for any other reduct B'.

Optimal: Given a decision table $S = (U, A \cup \{d\}, f)$, its discernibility matrix can be expressed as

$$m_{x,y} = \begin{cases} \{a \in A \mid f(x,a) \neq f(y,a)\}, & \text{if } f(x,d) \neq f(y,d) \\ \emptyset, & \text{otherwise} \end{cases} \tag{5}$$

where $x, y \in U$.

Based on the discernibility matrix, we construct the conjunctive normal form of discernibility information function as follows:

$$f_S = \wedge\{\vee B \mid \forall B \in \mathsf{Element}(m_{x,y})\}, \tag{6}$$

where $\mathsf{Element}(m_{x,y})$ is the set of all non-empty elements in the discernibility matrix $m_{x,y}$.

Lemma 2 *Let $S = (U, A \cup \{d\}, f)$ be a decision table and the discernibility information function of S is f_S, an attribute set $R \subseteq A$ is a reduct if and only if $\wedge R$ is the prime implicants of the Boolean function f_S.*

Similarly, if the disjunctive normal form of discernibility information function f_S is expressed as

$$f_S = \vee\{\wedge R \mid \forall R \in \mathsf{Red}(M)\}, \tag{7}$$

then $\mathsf{Red}(M)$ represents the set of all reducts [37].

Definition 3 Let $S = (U, A \cup \{d\}, f)$ be a decision table, let $B \subseteq A$ and $U' \subseteq U$. The ***core attributes*** in B over U' are defined as follows:

$$\mathsf{Core}^{U'}(B) = \{a \in B \mid \mathsf{POS}_{B \setminus \{a\}}^{U'}(d) \neq \mathsf{POS}_A^{U'}(d)\} \tag{8}$$

Again, when $U' = U$, we will just write $\mathsf{Core}(B)$ instead of $\mathsf{Core}^U(B)$. Note that if B is an attribute reduct, then $\mathsf{Core}(B) = B$; however, $\mathsf{Core}(B) = B$ does not always imply that B is an attribute reduct.

4 Vertex covers and attribute reducts

In this section, we will analyze the relationship between vertex covers of hypergraphs and attribute reducts of decision tables. This relationship is based on the concept of *induced decision tables*. Induced decision tables derived from graphs have been introduced by Chen in [36], its core idea is to obtain the decision table whose discernibility information function is equivalent to the Boolean function of the graph.

Let $G = (V, E)$ be a hypergraph with vertex set $V = \{v_1, v_2, \ldots, v_n\}$ and hyperedge set $E = \{e_1, e_2, \ldots, e_m\}$. The *incidence matrix of* G is a matrix $M_G = (m_{ij})_{m \times n}$, where

$$m_{ij} = \begin{cases} 1 & \text{if the hyperedge } e_i \text{ and the vertex } v_j \text{ are incident} \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

Definition 4 (Induced decision table from hypergraph). Let $G = (V, E)$ be a hypergraph with $V = \{v_1, v_2, \ldots, v_n\}$, $E = \{e_1, e_2, \ldots, e_m\}$ and $M_G = (m_{ij})_{m\times n}$. We define the ***induced decision table*** of G, denoted S_G, as $S_G = (U, A \cup \{d\}, f)$, where $U = E \cup \{e_{m+1}\}$ for some $e_{m+1} \notin E$, $A = V$, $d \notin A$, $\mathcal{V}_d = \{0, 1\}$ and the function f satisfies the following properties:

1. $1 \le i \le m \Rightarrow (f(e_i, d) = 1 \wedge \forall a \in A. f(e_i, a) = m_{i,j})$,
2. $i = m + 1 \Rightarrow \forall a \in A \cup \{d\}. f(e_i, a) = 0$.

We point out in particular that if G is known, then we will often write S instead of S_G.

Our approach is based on the following observations:

Lemma 3 (*Vertex Covers vs. Attribute Reducts* [37]). *Let $G = (V, E)$ be a hypergraph and $S_G = (E \cup \{e_{m+1}\}, V \cup \{d\}, f)$ be its induced decision table. Then we have*:

1. A set $B \subseteq V$ is a ***vertex cover*** of G if and only if it is a ***relative attribute reduct*** of S_G.
2. A set $B \subseteq V$ is a ***unique vertex cover*** of G if and only if it is an ***attribute reduct*** of S_G.
3. A set $B \subseteq V$ is a ***minimum vertex cover*** of G if and only if it is a ***minimal attribute reduct*** of S_G.

For completeness, as in this paper, we will only transform hypergraphs into decision tables, we will now show how to derive an appropriate hypergraph from a given decision table (with a single decision attribute).

Given a decision table $S = (U, A \cup \{d\}, f)$, we define $\mathcal{M}_S : U \times U \to 2^A$, *discredibility matrix* of S, as follows. For every $x, y \in U$, we have:

$$\mathcal{M}_S(x, y) = \begin{cases} \{a \in A \mid f(x, a) \neq f(y, a)\} & \text{if } f(x, d) \neq f(y, d) \\ \emptyset & \text{otherwise} \end{cases} \tag{10}$$

We will now define G_S as a *hypergraph* derived from S, i.e., $G_S = (V, E)$ where $V = A$ and $E = \{\mathcal{M}_S(x, y) \mid x, y \in U \wedge \mathcal{M}_S(x, y) \neq \emptyset\}$.

Lemma 4 (*Attribute Reducts vs. Vertex Covers* [37]). *Let $S = (U, A \cup \{d\}, f)$ be a decision table and $G_S = (V, E)$ the hypergraph derived from S. Then, we have*:

1. A set $B \subseteq V$ is a ***relative attribute reduct*** of S if and only if it is a ***vertex cover*** of G_S.
2. A set $B \subseteq V$ is a ***attribute reduct*** of S if and only if it is a ***unique vertex cover*** of G_S.
3. A set $B \subseteq V$ is a ***minimal attribute reduct*** of S if and only if it is a ***minimum vertex cover*** of G_S.

The discernibility information function of the decision table and the Boolean function of the hypergraph are equivalent by a particular transformation, then attribute reduction problem and vertex cover problem can be can be interchangeable. In other words, lemmas 3 and 4 allow us to use algorithms design for finding minimal attribute reduction (exact or approximate) for solving MVCP, and vice versa.

Immediately from Definition 4, we have that for each $S_G = (U, A \cup \{d\}, f)$,we have $U/d = \{\{e_1, e_2, \ldots, e_m\}, \{e_{m+1}\}\} = \{E, \{e_{m+1}\}\}$

Proposition 1 *Let $S_G = (U, A \cup \{d\}, f)$ be an induced decision table of G, $B \subseteq A$ and $e_{m+1} \in U' \subseteq U$. Then, we have:*

$$\mathsf{POS}_B^{U'}(d) = \begin{cases} U' & \text{if } |[e_{m+1}]_B| = 1 \\ U' \setminus [e_{m+1}]_B & \text{otherwise} \end{cases} \tag{11}$$

$$\mathrm{Core}^{U'}(B) = \bigcup_{e \in U'} \left\{ a \in B \middle| \sum_{b \in B} f(e, b) = 1 \wedge f(e, a) = 1 \right\} \tag{12}$$

Proof (1) Clearly, $U'/d = \{\{U' \setminus [e_{m+1}]_B\}, \{[e_{m+1}]B\}\}$. If $|[e_{m+1}]_B| = 1$, then $U/B = \{\{U' \setminus \{e_{m+1}\}\}, \{e_{m+1}\}\}$. Hence, $\mathsf{POS}_B^{U'}(d) = U'$. If $|[e_{m+1}]_B| \neq 1$, then $|[e_{m+1}]_B/d| \neq 1$. This means that for each $e \in [e_{m+1}]_B$, we have $e \notin \mathsf{POS}_B^{U'}(d)$, i.e., $\mathsf{POS}_B^{U'} = U' \setminus [e_{m+1}]_B$.

(2) Let $e \in U'$, $a \in B$, $\sum_{b \in B} f(e, b) = 1$ and $f(e, a) = 1$. Then, clearly $e \in [e_{m+1}]_{B\setminus\{a\}}$, i.e., $\mathsf{POS}_{B\setminus\{a\}}(d) \neq \mathsf{POS}_B(d)$. Therefore, $a \in \mathsf{Core}^{U'}(B)$.

Proposition 1 allows us to design of two efficient algorithms, one for a positive region $\mathsf{POS}_B^{U'}(d)$ and another for core attributes $\mathsf{Core}^{U'}(B)$. These two algorithms will later be used as parts of our main algorithms.

Algorithm 1 *Computing positive region* $\mathsf{POS}_B^{U'}(d)$

Input: Induced decision table $S_G = (U, A \cup \{d\}, f)$, $B \subseteq A$ and $e_{m+1} \in U' \subseteq U$.
Output: Positive region $\mathsf{POS}_B^{U'}(d)$.

```
Initialize unpos = ∅ ;
for each e ∈ U' do
    if Σ_{b∈B} f(e, b) = 0 then
        unpos = unpos ∪ {e} ;
    end if
end for
if |unpos| = 1 then
    POS_B^{U'}(d) = U'
else
    POS_B^{U'}(d) = U' \ unpos ;
end if
return POS_B^{U'}(d) ;
```

Algorithm 2 *Computing core attributes* $\mathsf{Core}^{U'}(B)$

Input: Induced decision table $S_G = (U, A \cup \{d\}, f)$, $B \subseteq A$ and $U' \subseteq U$, where $e_{m+1} \in U'$.
Output: Core attributes $\mathsf{Core}^{U'}(B)$.

Initialize $u = \emptyset$ and $\mathsf{Core}^{U'}(B) = \emptyset$;
for each $e \in U'$ **do**
  if $\sum_{b \in B} f(e, b) = 1$ **then**
    $u = u \cup \{e\}$;
  end if
end for
for each $a \in B$ **do**
  if $\sum_{e \in u} f(e, a) \neq 0$ **then**
    $\mathsf{Core}^{U'}(B) = \mathsf{Core}^{U'}(B) \cup \{a\}$;
  end if
end for
return $\mathsf{Core}^{U'}(B)$;

It is easy to see that the time complexity of Algorithm 1 is $O(|U'|)$ and the time complexity of Algorithm 2 is $O(|U'| + |B|)$.

5 Rough sets and local search (RS-LS)

This section describes in detail our local search method for solving the attribute reduction problem. This method will be then used to solve MVCP for a given hypergraph. [38]

5.1 Main Structure of RS-LS

The main idea of our approach is a simple consequence of the following obvious but important fact.

Proposition 2 *Let $S = (U, A \cup \{d\}, f)$ be a decision table, a relative attribute reduct B provides that the upper bound of the size of the minimal attribute reduct is $|B|$.*

Proof Proposition 2 is clearly established. If B is an minimal attribute reduct of S, then the conclusion is established. If B is not the minimal attribute reduction in S, then the size of the minimal attribute reduct must be less than $|B|$. This completes the proof.

From Proposition 2, it follows that if we have a relative attribute reduct, then we also have some upper bound of the minimal reduct. Then, we decrease the upper bound by removing an attribute from the current reduct. The outcome may or may not be a reduct. If it is not a reduct, we swap attributes by choosing one attribute from the current candidate reduct and the other that does not belong to the current candidate reduct. If the result is a reduct, it has smaller, i.e., better upper bound. We continue this process as long as it is possible, the outcome is a relatively small reduct or even an minimal attribute reduct. This scheme is shown in Fig. 2.

To start our process, we need firstly to initialize some reduct. We do it using *fast relative reduct construction method*. In each iteration of our method, we have to find a new relative reduct by swapping two appropriate attributes. To make this swapping efficient, we apply the *reverse incremental verification mechanism*, which verifies quickly whether the changed attribute pair meets the necessary requirements. Finally, to obtain additional speed up, *adjustment strategy* has been proposed.

All these three improvements are described in detail in the following subsections.

5.2 Fast relative reduct construction method

Algorithm 3, called *PosGreedyRed*, implements our fast relative reduct construction, which produces *some* reduct and starts iterations.

Algorithm 3 *Function PosGreedyRed*

Input: The decision table $S = (U, A \cup \{d\}, f)$.
Output: A relative attribute reduct B.

initialize $B = \emptyset$;
compute $unpos = \mathsf{POS}_A(d)$;
while $unpos \neq \emptyset$ **do**
  $B = B \cup \{a'\}$, where a' satisfies that $|\mathsf{POS}^{unpos}_{\{a'\}}(d)|$ is the biggest one ;
  $unpos = unpos \setminus \mathsf{POS}^{unpos}_{\{a'\}}(d)$;
end while
$temp = B \setminus \mathsf{Core}(B)$;
while $temp \neq \emptyset$ **do**
  select an attribute a^* from $temp$, where a^* satisfies that $|\mathsf{POS}_{\{a^*\}}(d)|$ is the smallest one ;
  if $\mathsf{POS}_A(d) = \mathsf{POS}_{B \setminus \{a^*\}}(d)$ **then**
    $B = B \setminus \{a^*\}$;
    $temp = B - \mathsf{Core}(B)$;
  else
    $temp = temp \setminus \{a^*\}$;
  end if
end while
return B ;

Algorithm 3 consists of two processes, one is the process of constructing a relative reduct, and the other one is the process of deleting redundant attributes. In the first process, the set B is initially empty. Then, in each iteration, we choose an attribute $a' \in A \setminus B$, such that $|\mathsf{POS}^{unpos}_{\{a'\}}(d)|$ has the biggest value. Then, we add the selected attribute a' to B. The greedy process stops when $unpos = \emptyset$, and then we have $\mathsf{POS}_B(d) = \mathsf{POS}_A(d)$. Therefore, the time complexity of this process is $O(\sum_{i=1}^{|B|} |unpos_i|)$, where $unpos_i$ represents the positive region associated with the i^{th} iteration. The process always converges, the worst case is when $B = A$. In the i^{th} iteration process, we know that $unpos_{i+1} = unpos_i \setminus \mathsf{POS}^{unpos_i}_{\{a'_i\}}(d)$, i.e., $|unpos_{i+1}| \leq |unpos_i|$. The worst-case time complexity is $O(\sum_{i=1}^{|A|} |unpos_i|)$, and it is usually much smaller than $O(|A||U|)$. The second process, at worst, accesses each attribute in B once. Hence, the worst-case time complexity is $O(|B||U|)$.

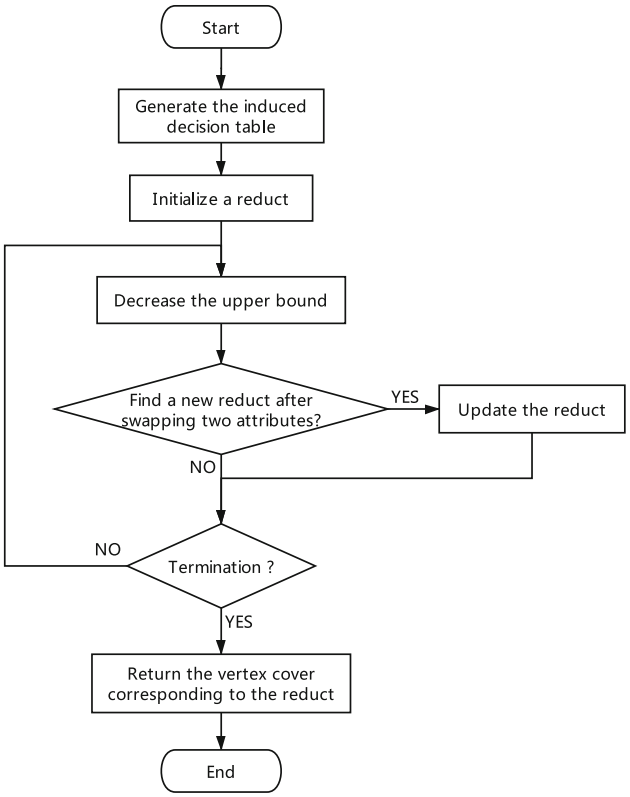


Fig. 2 Basic flowchart of our approach

5.3 (*p*, *q*)-reverse incremental verification mechanism

Suppose $S = (U, A \cup \{d\}, f)$ is a decision table, $B \subseteq C$ is a relative reduct, $a \in B$ and $B \setminus \{a\}$ is *not* a relative reduct. In such a case finding, a minimal attribute reduct gives rise to the following problem.

Problem 1 Let $S = (U, A \cup \{d\}, f)$ be a decision table, $B \subseteq A$ and $\mathsf{POS}_B(d) \neq \mathsf{POS}_A(d)$. How to select attributes p and q such that $\mathsf{POS}_{(B\setminus\{p\})\cup\{q\}}(d) = \mathsf{POS}_A(d)$?

We will use a reverse incremental verification mechanism to solve this problem and start with two useful lemmas.

Lemma 5 *Let $S = (U, A \cup \{d\}, f)$ be a decision table. For $B \subseteq A$, we have*: $\mathsf{POS}_B(d) = \mathsf{POS}_B^{\mathsf{POS}_B(d)}(d)$.

Proof Clearly, $\mathsf{POS}_B^{\mathsf{POS}_B(d)}(d) \subseteq \mathsf{POS}_B(d)$. Let $x \in \mathsf{POS}_B(d)$. Then, $x \in [x]/d \in \mathsf{POS}_B(d)/d$. But by the definition: $\mathsf{POS}_B^{\mathsf{POS}_B(d)}(d) = \bigcup_{X \in \mathsf{POS}_B(d)/d} B_-(X)$, hence $x \in \mathsf{POS}_B^{\mathsf{POS}_B(d)}(d)$.

Lemma 6 deals with increasing and decreasing given positive region of d.

Lemma 6 *Let $S = (U, A \cup \{d\}, f)$ be a decision table, and let $B \subseteq A$. Then, we have*:

1. For each attribute set $Q \subseteq A \backslash B$, $\mathrm{POS}_{B\cup Q}(d) = \mathrm{POS}_B(d) \cup \mathrm{POS}_Q^{U'}(d)$, where $U' = \mathsf{POS}_{B\cup Q}(d) \setminus \mathsf{POS}_B(d)$.
2. For each attribute set $P \subseteq B$, $\mathsf{POS}_{B\setminus P}(d) = \mathsf{POS}_B(d) \setminus \bigcup_{X\in\mathcal{X}_p} X$, where $\mathcal{X}_P = \{X \mid X \in \mathsf{POS}_B(d)/\mathsf{ind}(B) \wedge (X \times U) \cap \mathsf{iop}(B \setminus P) \neq \emptyset\}$.

Proof (1) First note that $U' \cap \mathsf{POS}_B(d) = \emptyset$ and $\mathsf{POS}_Q^{U'}(d) \subseteq U'$, so $\mathsf{POS}_{B\cup Q}(d) = \mathsf{POS}_B(d) \cup \mathsf{POS}_Q^{U'}(d) \Longleftrightarrow U' = \mathsf{POS}_{\{Q\}}^{U'}(d)$. Suppose that $x \in U' \setminus \mathsf{POS}_Q^{U'}(d)$, i.e., $x \in \mathsf{POS}_{B\cup Q}(d)$, $x \notin \mathsf{POS}_B(d)$ and $x \notin \mathsf{POS}_Q^{U'}(d)$, which clearly implies $[x]_{\mathsf{ind}(B\cup Q)} \subseteq \mathsf{POS}_{B\cup Q}(d)$, $[x]_{\mathsf{ind}(B)} \cap \mathsf{POS}_B(d) = \emptyset$ and $[x]_{\mathsf{ind}(Q)} \cap \mathsf{POS}_Q^{U'}(d) = \emptyset$. However, since $Q \cap B = \emptyset$, we also have $\mathsf{ind}(B \cup \{v\}) = \mathsf{ind}(B) \cap \mathsf{ind}(\{v\})$, which means $[x]_{\mathsf{ind}(B\cup Q)} \subseteq [x]_{\mathsf{ind}(B)} \cap [x]_{\mathsf{ind}(Q)}$, a contradiction.

(2) Since $B \setminus P \subsetneq B$, then $\mathsf{POS}_{B\setminus P}(d) \subseteq \mathsf{POS}_B(d)$. Consider $X \in \mathcal{X}_P$. Since $X \in \mathsf{POS}_B(d)/\mathsf{ind}(B)$, then $X \subseteq \mathsf{POS}_B(d)$, and since $(X \times U) \cap \mathsf{iop}(B \setminus P) \neq \emptyset$, then $X \cap \mathsf{POS}_{B\setminus P}(d) = \emptyset$. Hence, $\mathsf{POS}_{B\setminus P}(d) \subseteq \mathsf{POS}_B(d) \setminus \bigcup_{X\in\mathcal{X}_p} X$. Let $x \in \mathsf{POS}_B(d) \setminus \bigcup_{X\in\mathcal{X}_p} X$. Hence, $x \in \mathsf{POS}_B(d)$, and there is $y \in U$ such that $(x, y) \notin \mathsf{iop}(B \setminus P)$, i.e., $x \in \mathsf{POS}_{B\setminus P}(d)$.

Lemma 6 shows the results of adding and deleting sets of attributes to and from a positive region $\mathsf{POS}_B(d)$. We will use them to implement the reverse incremental verification mechanism, as shown below.

Proposition 3 *Let $S = (U, A \cup \{d\}, f)$ be a decision table $P \subseteq B \subsetneq A$ and $Q \subseteq A \setminus B$ such that*

- $\mathsf{POS}_B(d) \neq \mathsf{POS}_A(d)$,
- $\mathsf{POS}_{(B\setminus P)\cup Q}(d) = \mathsf{POS}_A(d)$,
- $\mathsf{POS}_A(d)/\mathsf{ind}(B \cup Q) = \{X_1, \ldots, X_n\}$.

Then, the following properties hold:

1. $\mathsf{POS}_Q^{U'}(d) = U'$, where $U' = \mathsf{POS}_A(d) \setminus \mathsf{POS}_B(d)$,
2. $\mathsf{POS}_{(B\setminus P)\cup Q}^{\widehat{U}}(d) = \widehat{U}$, for every $\widehat{U} = \{x_1, \ldots, x_n\} \subseteq U$ such that $\widehat{U} \cap X_i = \{x_i\}$ for $i = 1, \ldots, n$,
3. $P \cap \mathsf{Core}^{\widehat{U}}(B \cup Q) = \emptyset$.

Proof (1) Since $\mathsf{POS}_{(B\setminus P)\cup\{Q\}}(d) = \mathsf{POS}_A(d)$, then we have: $\mathsf{POS}_{B\cup Q}(d) = \mathsf{POS}_A(d)$. By Lemma 6(1) we have: $\mathsf{POS}_{B\cup Q}(d) = \mathsf{POS}_B(d) \cup \mathsf{POS}_Q^{\mathsf{POS}_A(d)\setminus\mathsf{POS}_B(d)}(d)$, i.e., $\mathsf{POS}_Q^{\mathsf{POS}_A(d)\setminus\mathsf{POS}_B(d)}(d) = \mathsf{POS}_A(d) \setminus \mathsf{POS}_B(d)$.

(2) From Lemma 5, it follows $\mathsf{POS}_{(B\setminus P)\cup Q}(d) = \mathsf{POS}_{(B\setminus P)\cup Q}^{\mathsf{POS}_{(B\setminus P)\cup Q}(d)}(d)$ and $\mathsf{POS}_{B\cup\{v\}}(d) = \mathsf{POS}_{B\cup\{v\}}^{\mathsf{POS}_{B\cup\{v\}}(d)}(d)$.However, $\mathsf{POS}_{(B\setminus P)\cup Q}(d) = \mathsf{POS}_{B\cup Q}(d) = \mathsf{POS}_A(d)$,so $\mathsf{POS}_{(B\setminus P)\cup Q}^{\mathsf{POS}_A(d)}(d) = \mathsf{POS}_{B\cup Q}^{\mathsf{POS}_A(d)}(d)$.On the other hand, since $(B\setminus P)\cup Q = (B\cup Q)\setminus P$,by Lemma 6(2), we have $\mathsf{POS}_{(B\setminus P)\cup Q}^{\mathsf{POS}_A(d)}(d) = \mathsf{POS}_{B\cup Q}^{\mathsf{POS}_A(d)}(d)\setminus\bigcup_{X\in\mathcal{X}_P^Q} X$ where $\mathcal{X}_P^Q = \{X \mid X\in \mathsf{POS}_A(d)/\mathsf{ind}(B\cup Q)\wedge(X\times \mathsf{POS}_A(d))\cap \mathsf{iop}((B\cup Q)\setminus P)\neq\emptyset\}$. This means that $\bigcup_{X\in\mathcal{X}_P^Q} X = \emptyset$, i.e., $\mathcal{X}_P^Q = \emptyset$, or, equivalently, $X\in \mathsf{POS}_A(d)/\mathsf{ind}(B\cup P)$ implies $(X\times \mathsf{POS}_A(d))\cap \mathsf{iop}((B\cup Q)\setminus P) = \emptyset$. But this also means that $X\in \mathsf{POS}_A(d)/\mathsf{ind}(B\cup Q) = \{X_1,\ldots,X_n\}$ implies $X\subseteq \mathsf{POS}_{(B\setminus P)\cup Q}^{\mathsf{POS}_A(d)}(d)$. For each $i = 1,\ldots,n$, let x_i be an arbitrary element of X_i and set $\widehat{U} = \{x_1,\ldots,x_n\}$. If $i\neq j$, then we now have $(x_i,x_j)\in \mathsf{ind}((B\setminus P)\cup Q)$ and $f(x_i,d) = f(x_j,d)$. But this means that we have $\mathsf{POS}_{(B\setminus P)\cup Q}^{\widehat{U}}(d) = \widehat{U}$.

(3) Since $\mathsf{POS}_{(B\setminus P)\cup Q}^{\widehat{U}}(d) = \widehat{U}$ for any $P\subseteq B$, including $P = \emptyset$, we have $\mathsf{POS}_{(B\setminus P)\cup Q}^{\widehat{U}}(d) = \mathsf{POS}_{B\cup Q}^{\widehat{U}}(d)$. Suppose that $p\in P\cap \mathsf{Core}^{\widehat{U}}(B\cup Q)$. From Definition 3, we have $\mathsf{POS}_{(B\setminus P)\cup Q}^{\widehat{U}}(d)\neq \mathsf{POS}_{B\cup Q}^{\widehat{U}}(d)$, a contradiction, i.e., $P\cap \mathsf{Core}^{\widehat{U}}(B\cup Q) = \emptyset$.

A special case of Proposition 3, when $P = \{p\}$ and $Q = \{q\}$, provides basis for a solution to Problem 1, so we provide its precise formulation as below.

Corollary 1 *Let $S = (U, A\cup\{d\}, f)$ be a decision table $p\in B\subsetneq A$ and $q\in A\setminus B$ such that*

- $\mathsf{POS}_B(d)\neq \mathsf{POS}_A(d)$,
- $\mathsf{POS}_{(B\setminus\{p\})\cup\{q\}}(d) = \mathsf{POS}_A(d)$,
- $\mathsf{POS}_A(d)/\mathsf{ind}(B\cup\{q\}) = \{X_1,\ldots,X_n\}$.

Then, the following properties hold:

1. $\mathsf{POS}_{\{q\}}^{U'}(d) = U'$, where $U' = \mathsf{POS}_A(d)\setminus \mathsf{POS}_B(d)$,
2. $\mathsf{POS}_{(B\setminus\{p\})\cup\{q\}}^{\widehat{U}}(d) = \widehat{U}$, for every $\widehat{U} = \{x_1,\ldots,x_n\}\subseteq U$ such that $\widehat{U}\cap X_i = \{x_i\}$ for $i = 1,\ldots,n$,
3. $p\in B\setminus \mathsf{Core}^{\widehat{U}}(B\cup\{q\})$

Corollary 1 suggests the following useful definition. Let $S = (U, A\cup\{d\}, f)$ be a decision table, $B\subsetneq A$ and $U' = \mathsf{POS}_A(d)\setminus \mathsf{POS}_B(d)$. We define $A_B^*\subseteq A$, a ***set of attributes filtered by*** B as:

$$A_B^* = \{q \mid q\in A\setminus B\wedge \mathsf{POS}_q^{U'}(d) = U'\}. \tag{13}$$

5.4 Adjustments strategies

From the basic flowchart of our approach in Fig. 1, we need to set the termination condition and the method of decreasing the upper bound. The general iterative algorithm usually stops after a certain number of iterations, but it will produce a lot of redundant computation when the algorithm falls into local optimal. At the same time, we should avoid repeated computation and premature falling into local optimum in the iterative process. To design a more effective algorithm, the adjustment strategies for the algorithm related processes are shown in Table 2.

These adjustment strategies in Table 2 bring the following benefits:

1. In the process of decreasing the upper bound, it avoids deleting the attributes that have been deleted in the previous iteration.
2. Obviously, $RemoveSet\subseteq Red$ and $RemoveSet = Red$ means that no matching attribute pairs can be found after an attribute removed from Red. That is to say, we found a local optimal solution. Therefore, setting $|RemoveSet|\neq|Red|$ as a termination condition can avoid invalid redundant calculations.
3. T is a safety precaution. Because (p, q)-reverse incremental verification mechanism is actually a greedy search process, we also need to set the maximum number of iterations. The safety precaution ensures that the algorithm outputs the results in a certain period of time.

5.5 Implementation of RS-LS

Algorithm 4, which implements RS-LS procedure, applies all techniques describes in previous subsections. According to the above description, the pseudo-code of the novel

Table 2 Adjusting strategies

Related processes	Detailed strategies
Set an attribute set *RemoveSet*	Records the currently deleted attribute set
Decrease the upper bound	Randomly deleted an attribute $a \in Red \setminus RemoveSet$
Initialization of *RemoveSet*	$RemoveSet = \emptyset$
The attribute pair is found	$RemoveSet = \emptyset$
The attribute pair can't be found	$RemoveSet = RemoveSet \cup \{a\}$
Termination conditions	$t < T$ and $\|RemoveSet\| \neq \|Red\|$

rough set-based approach for minimum vertex cover problem of hypergraph is described in Algorithm 4.

Algorithm 4 *RS-LS*

Input: A hypergraph $G = (V, E)$, the maximum iterations T.
Output: A vertex cover R of the hypergraph G.

1: generate a induced decision table $S = (U, A \cup \{d\}, f)$ from the hypergraph G ;
2: initialize $t = 0$ and $RemoveSet = \emptyset$;
3: construct a relative reduct $Red = PosGreedyRed(S)$;
4: **while** $t < T \wedge |Red| \neq |RemoveSet|$ **do**
5: **if** $\mathsf{POS}_{Red}(d) = \mathsf{POS}_{Red \setminus \{a\}}(d)$ **then**
6: $Red = Red \setminus \{a\}$;
7: $temp = B - \mathsf{Core}(B)$;
8: **else**
9: **if** $(p, q) := VerificationMechanism(S, Red \setminus \{a\}) \neq (0, 0)$ **then**
10: $Red = Red \setminus \{a, p\} \cup \{q\}$;
11: $RemoveSet = \emptyset$;
12: **else**
13: $RemoveSet = RemoveSet \cup \{a\}$;
14: **end if**
15: $t = t + 1$;
16: **end if**
17: **end while**
18: **return** R ;

Before analyzing the time complexity of Algorithm 4, we first analyze the time complexity of Algorithm 5, which computes the function *VerificationMechanism* and is used inside Algorithm 4.

Algorithm 5 Function *VerificationMechanism*

Input: The decision table $S = (U, A \cup \{d\}, f)$ and current candidate attribute set B.
Output: A pair of attributes (p, q).

1: initialize $A_B^* = \emptyset$, $p = 0$, and $q = 0$;
2: construct a set $U' = \mathsf{POS}_A(d) \setminus \mathsf{POS}_B(d)$;
3: **for** each $a \in A \setminus B$ **do**
4: **if** $\mathsf{POS}_{\{a\}}^{U'}(d) = U'$ **then**
5: $A_B^* = A_B^* \cup \{a\}$, which filters the attribute set A_B^* according to condition (1) in Proposition 3
6: **end if**
7: **end for**
8: **for** each $q' \in A_B^*$ **do**
9: compute $\mathsf{POS}_A(d)/\mathsf{ind}(B \cup \{q'\}) = \{X_1, \ldots, X_n\}$;
10: construct a set $\widehat{U} = \{x_1, \ldots, x_n\}$, where $x_i \in X_i$;
11: compute the core attributes in $B \cup \{q'\}$ over $\widehat{U}$ by using Algorithm 2, i.e., $\mathsf{Core}^{\widehat{U}}(B \cup \{q'\})$;
12: **for** each $p' \in B \setminus \mathsf{Core}^{\widehat{U}}(B \cup \{q'\})$ **do**
13: **if** $\mathsf{POS}_{B \setminus \{p'\} \cup \{q'\}}(d) = \widehat{U}$ **then**
14: $p = p'$ and $q = q'$;
15: **break**;
16: **end if**
17: **end for**
18: **end for**
19: **return** (p, q) ;

The function *VerificationMechanism* verifies whether a proper attribute pair (p, q) can be found and exchanged. Step 2 constructs a set $U' = \mathsf{POS}_A(d) \setminus \mathsf{POS}_B(d)$, and the time complexity is $O(|U|)$. Steps 3-7 find the filtered attribute set A_B^*, and the time complexity of this process is $O(|A||U'|)$. The time complexity of steps 8-18, i.e., selecting q and p from A_B^* and $B \setminus \mathsf{Core}^{\widehat{U}}(B \cup \{q\})$, is $O(|A_B^*||B \setminus \mathsf{Core}^{\widehat{U}}(B \cup \{q\})||\widehat{U}|)$.

Therefore, the *worst* time complexity of Algorithm 5 is $O(|U| + |A||U'| + |A_B^*||B - \mathsf{Core}^{\widehat{U}}(B \cup \{q\})||\widehat{U}|) = O(|A|^2|U|) = O(|V|^2|E|)$.

However, since usually $|\widehat{U}| \ll |U|$, $|A_B^*| \ll |A|$ and $|B \setminus \mathsf{Core}^{\widehat{U}}(B \cup \{q\})| < |B| \ll |A|$, an average case time complexity of Algorithm 5 is usually much smaller than $O(|A|^2|U|) = O(|V|^2|E|)$. Actually, our tests indicate that an average time of Algorithm 5 is often even smaller than $O(|U|) = O(|E|)$.

In Algorithm 4, step 1 generates a induced decision table, and the time complexity is $O(|V| + |E|)$. Step 3 constructs a relative reduct using Algorithm 3, so the worst time complexity of this step is $O(|B||U|) = O(|A||U|) = O(|V||E|)$.

For the time essential steps inside the loop while do, let Red_i represents the relative reduct used in the i^{th} iteration. If $\mathsf{POS}_{Red_i}(d) = \mathsf{POS}_{Red_i \setminus \{a\}}(d)$, then the time complexity of the i^{th} iteration is $O(|U|)$. If not, we need to run the function $(p, q) = VerificationMechanism(S, Red_i \setminus \{a\})$, i.e., Algorithm 5, which is $O(|A|^2|U|)$ for the worst case and $O(|U|)$ in estimated average case.

The loop while do executes either T times (safety precautions) or $O(|Red|) = O(|A|)$ times, whichever comes first, so the worst time complexity is $O(\min(T, |A|)|A|^2|U|) = O(\min(T, |V|)|V|^2|E|)$; however, an expected average complexity is usually $O(\min(T, |V|)|U|)$.

Table 3 is the comparison of time complexities (worst time complexity and estimated average time complexity) for computing the minimum vertex cover of a hypergraph $G = (V, E)$ by using different rough sets-based methods.

Table 3 Complexity comparisons

Methods	Worst Time	Average Time
VCRS [30]	$O(\|V\|\|E\|^2)$	$O(\|V\|\|E\|^2)$
VCAR [36][a]	$O(\|V\|\|E\| + \sum_{i=1}^{\|V\|} \|E_i\|)$	$O(\|V\|\|E\|)$
IFS-S [37]	$O(\|V\|^2\|E\|)$	$O(\|V\|^2\|E\|)$
IFS-M [37][b]	$O(K\|V\|\|E\|)$	$O(\|V\|\|E\|)$
SPS-G [39]	$O(\|V\|^2\|E\|)$	$O(\|V\|^2\|E\|)$
RS-LS[c]	$O(\min(T, \|V\|)\|V\|^2\|E\|)$	$O(\min(T, \|V\|)\|E\|)$

[a]In VCAR, E_i represents the edge set associated with i^{th} iteration.

[b]K is a given parameter in IFS-M, i.e., dividing decision table into K groups by attributes ($K = 10$ was used).

[c]T is the maximum number of iterations in RS-LS ($T = 2000$ was used)

6 Experiments

The goal of carried out experiments is to verify the efficiency of our RS-LS algorithm. All the experiments were performed on a personal computer with Windows 10 and Intel (R) Core (TM) i5-7300HQ CPU 2.50 GHz and 16.0 GB memory. The algorithms were implemented in MATLAB 2017a. Our RS-LS was compared with other two rough set-based approaches for finding MVCP for hypergraphs. The first one was the general heuristic algorithm VCAR from [36], the other was dimension incremental attribute reduction algorithm IFS-S from [37]. The maximum iterations of our proposed RS-LS algorithm was set as $T = 2000$.

6.1 Results on general graphs

For our experiments, we select 40 benchmarks from the Network Data Repository [41]. Each algorithm runs 10 times on each graph, and we record the best vertex cover and the average computation time of the 10 runs. The results are shown in Table 4, the column *Value* represents the number of selected vertices the column *Time* represents the average running time. If the value is the best one, then we marked it in bold.

As shown in Table 4, we can observe that VCAR runs faster than IFS-S and RS-LS. However, it should emphasized that the algorithm VCAR returns larger size of the vertices than that of RS-LS on all graphs. On most graphs, VCAR is also worse than IFS-S in terms of the vertices size. IFS-S returns the best size of the vertices on graph soc-dolphins, but the running time of IFS-S is the worst one among these three algorithms. Our proposed RS-LS algorithm obtains the smallest size of the vertices, and its running time is faster than that of IFS-S. It is remarkable that there is no obvious difference between the running time of RS-LS and VCAR on most tested graphs.

6.2 Results on hypergraphs

For all these graphs shown in Table 4, one can see that the edges of the graphs have at most two vertices. However, our RS-LS algorithm has been designed for hypergraphs. To evaluate the performance of our algorithm on hypergraphs, we selected some hypergraphs from [42] and generated four hypergraphs from information tables Dermatology [43], CNAE-9 [43], Relathe [44], and Basehock [44].

To record the experimental results fairly, each algorithm also runs 10 times on each graph, and we record the best vertex cover and the average computation time of the 10 runs. The generated hypergraphs are shown in Table 5, and the results of all hypergraphs are shown in Table 6.

Table 6 gives the best vertex cover size and the average computation time of 10 runs on 20 hypergraphs, in which the best value of the vertex cover size is marked in bold. We can observe that our proposed algorithm RS-LS returns much smaller size of the vertex cover than that of VCAR and IFS-S, but its running time is slower than that of VCAR. However, it should emphasized that VCAR obtains the worst size of vertex cover among these three algorithms. IFS-S is worse than other two algorithms in terms of the running time, but it achieves smaller size of the vertex cover than that of VCAR on all hypergraphs except for epb1, lhr14, us04, ted_A, rim, G1 and G3.

Summing up, the experimental results on both graphs and hypergraphs show that our new RS-LS algorithm can achieve a smaller size of the vertex cover than the other two algorithms with a comparably computational time.

6.3 Efficiency of LS-RS

The efficiency of LS-RS is partially discussed in Sections 5.1 and 5.2. In this subsection, we conduct the experiments to examine the time consumption of LS-RS on random graphs where we respectively vary the sizes of edge set and vertex set.

We first explore the detailed change trend in computational time of LS-RS with the size of edge set increasing but the size of vertex set invariant. For this purpose, we generated two groups of random graphs. In the first group, the number of vertices is fixed to 200 and the number of edges is varied from 1000 to 10,000. The second group contains a fixed number of vertices for 2000 and a varying number of vertices from 5000 to 50 000. To focus on the time consumption of LS-RS with different number of vertex set, we also generate two groups of random graphs. One group contains a fixed number of edges for 40,000 and

Table 4 Comparison of three rough sets-based algorithms on graphs

Graphs	$\|V\|$	$\|E\|$	VCAR		IFS-S		RS-LS	
			Value	Time	Value	Time	Value	Time
bio-grid-mouse	1455	3272	428	8.739	427	238.72	**426**	17.735
bio-yeast	1458	1948	464	6.842	462	207.474	**457**	15.746
bio-celegans-dir	453	4596	260	2.844	259	276.849	**253**	10.133
ca-CSphd	1882	1740	555	9.529	553	364.951	**550**	14.366
econ-mahindas	1258	7682	752	52.908	746	447.237	**742**	124.095
email-dnc-core	2029	1,2085	412	25.327	412	681.261	**411**	143.011
email-enron-only	143	623	88	0.157	88	1.155	**86**	0.416
email-univ	1133	5451	605	19.147	602	262.947	**598**	41.847
ia-fb-messages	1266	6451	595	27.775	593	351.515	**285**	65.022
ia-infect-dublin	410	2765	298	2.56	296	16.053	**295**	6.946
ia-infect-hyper	113	2196	93	0.281	93	3.421	**90**	1.759
rt-retweet	96	117	33	0.032	33	0.137	**32**	0.086
rt-twitter-copen	761	1029	238	1.314	238	6.246	**237**	2.418
soc-dolphins	62	159	35	0.086	**34**	0.13	**34**	0.112
soc-wiki-Vote	889	2914	411	8.153	409	115.41	**408**	21.481
soc-hamsterster	2426	1,6630	1619	377.073	1619	2486.032	**1615**	442.816
130bit	584	6120	427	9.56	426	118.891	**418**	34.384
145bit	1002	1,1315	722	48.192	716	539.654	**708**	123.927
162bit	3606	3,7118	2562	1854.219	2547	1,1603.638	**2519**	1989.287
tech-routers-rf	2113	6632	807	66.243	804	279.707	**799**	190.235
frb30-15-1	450	1,7827	429	25.479	429	178.474	**424**	38.162
frb30-15-2	450	1,7874	431	29.676	429	220.47	**424**	41.253
frb30-15-3	450	1,7809	429	28.018	426	196.081	**423**	43.678
frb30-15-4	450	1,7831	429	27.348	429	217.473	**424**	38.206
frb30-15-5	450	1,7794	430	31.505	427	250.986	**423**	44.729
frb35-17-1	595	2,7856	570	70.103	567	486.895	**564**	80.725
frb35-17-2	595	2,7847	570	66.773	570	489.593	**564**	78.229
frb35-17-3	595	2,7931	571	67.765	568	470.495	**565**	76.892
frb35-17-4	595	2,7842	570	67.149	570	518.072	**566**	79.816
frb35-17-5	595	2,8143	571	68.813	569	510.881	**566**	84.522
frb40-19-1	760	4,1313	737	76.645	731	747.969	**726**	122.706
frb40-19-2	760	41262	732	86.499	731	722.77	**726**	112.777
frb40-19-3	760	4,1094	732	81.068	731	725.502	**726**	113.644
frb40-19-4	760	4,1604	732	85.776	730	709.375	**726**	117.482
frb40-19-5	760	4,1618	732	85.792	731	720.115	**726**	120.020
frb45-21-1	945	5,9185	916	117.822	914	1239.927	**907**	214.416
frb45-21-2	945	5,8624	916	112.920	913	1288.185	**907**	223.031
frb45-21-3	945	5,8244	917	107.488	912	1306.301	**907**	218.718
frb45-21-4	945	5,8548	913	113.633	912	1285.029	**909**	224.076
frb45-21-5	945	5,8578	917	107.488	913	1249.629	**908**	219.496

a varying number of vertices from 300 to 3000. In the other group, the number of edges is fixed to 10 000 and the number of vertices is varied from 1000 to 10,000. Each algorithm runs 10 times on each graph, and we record the average computation time of three algorithms and compare the average computation time of two internal processes (initialize a reduct and the loop while do) of LS-RS. The results are shown in Figs. 3, 4, 5 and 6.

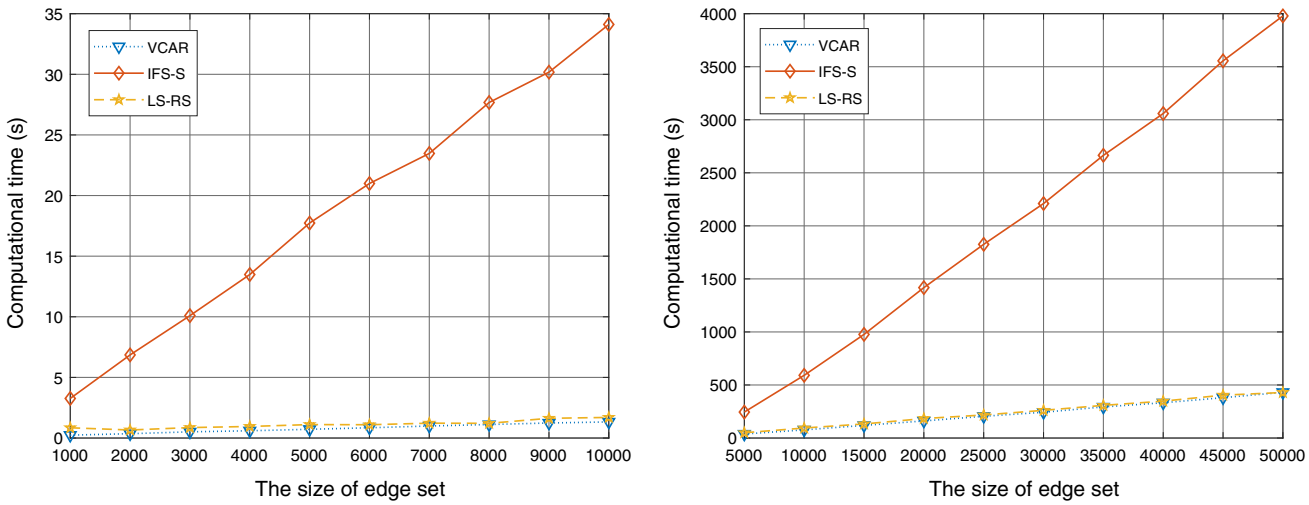
The detailed change trend in computational time of these three algorithms with the increase in the edge size is shown in Fig. 3. It is easy seen that VCAR and RS-LS are faster than IFS-S to obtain a minimum vertex cover. For example, the computational time of IFS-S is about nine

Table 5 Complexity comparisons

Numbers	Names	Areas	Objects	Attributes	Classes	Vertices	Hyperedges
G1	Dermatology	Biological	366	33	6	33	39,847
G2	CNAE-9	Business	1080	856	9	856	4,89,919
G3	Relathe	Text	1427	4322	2	4322	5,03,365
G4	Basehock	Text	1993	4862	2	4862	9,87,784

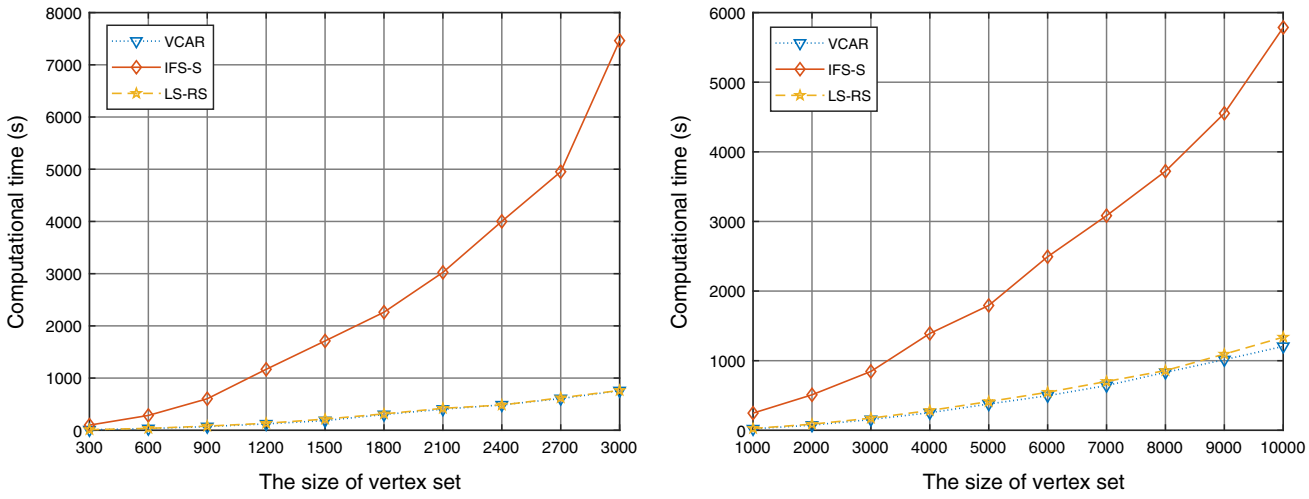
Table 6 Comparison of three rough sets-based algorithms on hypergraphs

Hypergraphs	$\|V\|$	$\|E\|$	VCAR		IFS-S		RS-LS	
			Value	Time	Value	Time	Value	Time
gemat1	10,595	4929	805	162.127	796	3451.454	**779**	234.148
cage10	11,397	11,397	1506	519.700	1501	2324.435	**1473**	652.691
coupled	11,341	11,341	1952	450.283	1924	6874.464	**1850**	582.862
crystk02	13,965	13,965	253	138.606	**252**	505.245	**252**	175.552
epb1	14,734	14,734	2470	1434.317	2470	2641.678	**2461**	1727.079
PGPgiantcompo	10,680	10,680	3037	234.488	3035	1992.723	**3032**	340.950
lhr14	14,270	14,270	2966	874.501	2966	3312.091	**2962**	1007.111
us04	28,016	163	24	2.755	24	33.682	**20**	6.238
ted_A	10,605	10,605	4309	33.985	4309	1544.777	**4290**	82.754
rim	22,560	22,560	1066	905.883	1066	3261.308	**1030**	1099.446
shermanACb	18,510	18,510	1900	633.008	1899	2015.024	**1895**	808.209
psse2	11,028	28,634	2853	2326.906	2835	8110.231	**2799**	2602.897
nopoly	10,774	10,774	2256	809.338	2216	4061.681	**2035**	939.295
TF16	19,321	15,437	2477	2219.682	2469	6645.672	**2438**	2568.888
ISPD98_ibm01	12,752	14,111	4768	2233.683	4708	1,1796.424	**4635**	2352.644
ISPD98_ibm02	19,601	19,584	7110	6261.739	6993	2,4103.259	**6914**	6488.443
G1	33	39,847	9	0.167	9	3.175	**8**	0.532
G2	856	4,89,919	76	55.968	75	1042.681	**71**	188.186
G3	4322	5,03,365	24	134.486	24	715.274	**23**	304.293
G4	4862	9,87,784	38	563.013	37	2412.079	**35**	941.064



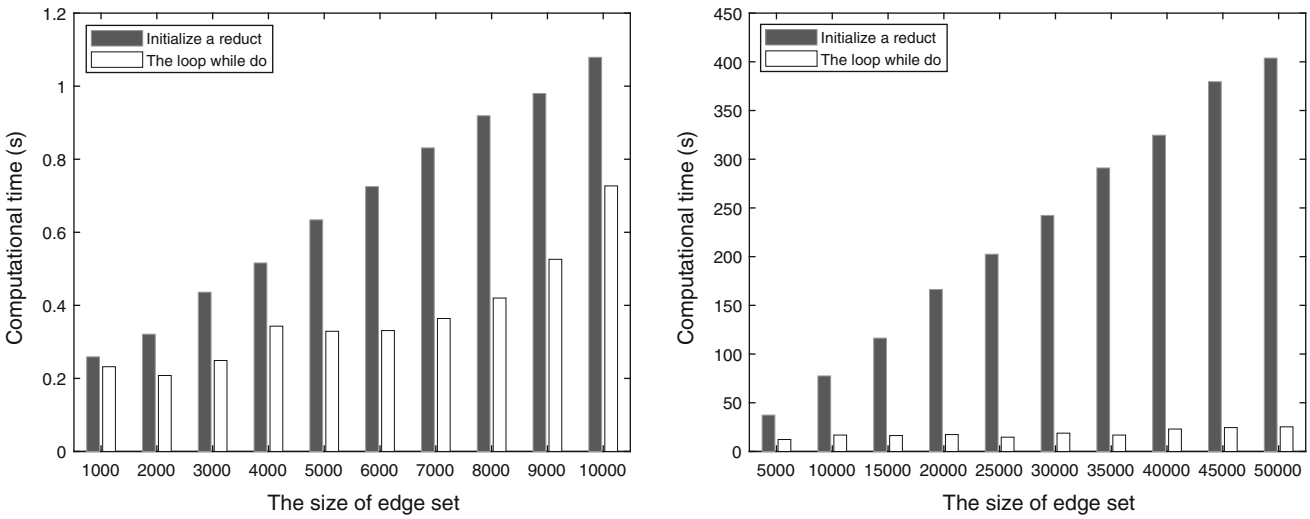
(a) The size of vertex set: 200 **(b)** The size of vertex set: 200

Fig. 3 Computational time comparison of three algorithms with different size of the edge set



(a) The size of edge set: 40 000 **(b)** The size of edge set: 10 000

Fig. 4 Computational time comparison of three algorithms with different size of the vertex set



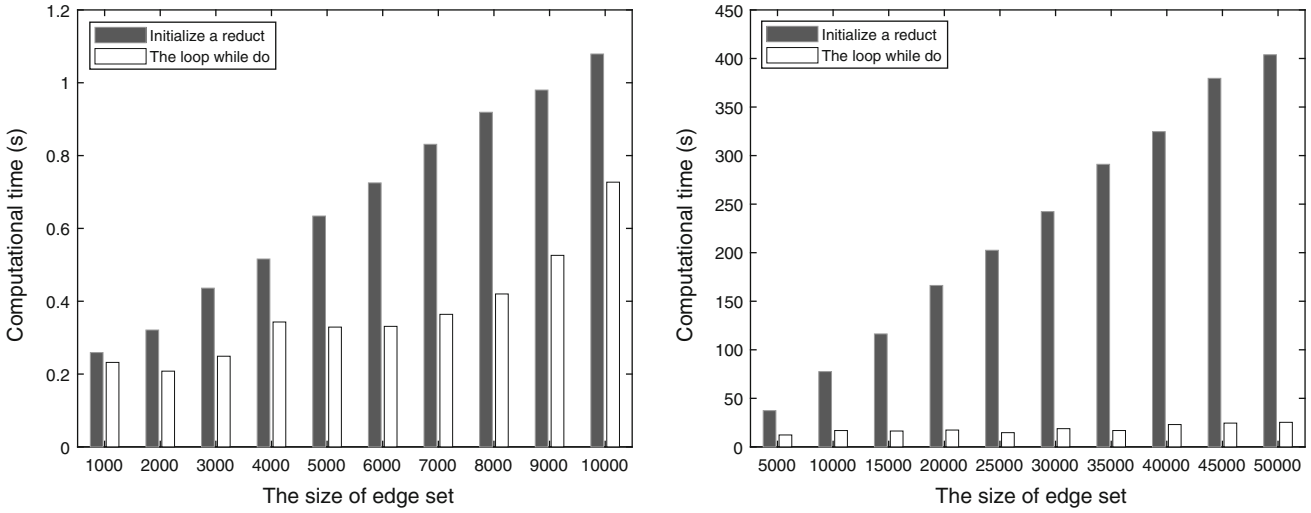
(a) The size of vertex set: 200 **(b)** The size of vertex set: 2 000

Fig. 5 Computational time comparison of two processes of LS-RS with different size of the edge set

times as much as that of RS-LS when the size of edge set is 50,000, and the size of vertex set is 2 000. It is worth noting that the differences between IFS with other two algorithms are more obvious when the size of edge set increases. We can also observe that the curves corresponding to these three algorithms approximately scale linearly to the number of edges. The reason is due to the complexity of these algorithms as we have analyzed previously. In addition, the performances of RS-LS and VCAR are almost the same when the size of edge set increases.

Figure 4 shows the curves of computational time of three tested algorithms with the number of vertices increasing. The trend of curves is similar to those shown in Fig. 3, but the increases for IFS-S is not linear to the size of vertices since its estimated average case time complexity is $O(|V|^2|E|)$. However, RS-LS and VCAR also scale linearly to the size of vertices. Similar to Fig. 3, there is no significant difference in computational time between RS-LS and VCAR as the number of vertices increases.

Figures 5 and 6 show the time consumption of the two internal processes of LS-RS with different size of edge set and vertex set, respectively. As shown in Figs. 5 and 6, we can see that LS-RS spends the majority of computational time in the process of initializing a reduct. With the increase in the size of edges or the size of vertices, the



(a) The size of edge set: 40 000 **(b)** The size of edge set: 10 000

Fig. 6 Computational time comparison of two processes of LS-RS with different size of the vertex set

process of initializing a reduct increases gradually. The reason for this observation is that its estimated average case time complexity is $O(|V||E|)$. More significantly, however, the computational time of the loop while do has not changed dramatically with the size of edges or the size of vertices increasing. For example, when the number of vertices is 2000, the first process time increases linearly with the increase in the number of vertices, but the second process consumes almost the same amount of computational time. This result validates why RS-LS has comparable running time efficiency compared with VCAR.

7 Conclusion

A new heuristic algorithm for solving the minimum vertex cover problem for hypergraphs was provided and tested. Our solution is based on the fact that minimum vertex problem for hypergraphs and minimal attribute reducts for rough sets decision tables are mutually related and both employ a local search paradigm.

Rough sets-based methods for solving the minimum vertex cover problem for hypergraphs have been proposed before; however, all these techniques have often found solutions that were far away from optimal ones. On the other hand, local search-based techniques have been shown to be an effective and promising approach to solve minimum vertex cover problem of graphs, but they often failed for hypergraphs.

We believe our approach combines good aspects of both rough set theory and the local search. Our solution is a stochastic local search algorithm for minimal attribute reduct and has embedded three advanced improvements to enhance its efficiency, i.e., fast relative reduct construction, reverse incremental verification and adjusting strategy.

The results of the experiments on benchmark graphs and hypergraphs indicated that our algorithm significantly outperforms the state-of-the-art algorithms in terms of the vertex cover size. Moreover, it has a comparable computation time.

Acknowledgements The authors thank the support by Nanjing University of Posts and Telecommunications. The work is also sponsored by the National Natural Science Foundation of China(-Grant Nos. 61902199 and 61872197); NUPTSF (Grant No. NY219142); the Research start up Foundation of Nanjing Agricultural University (Grant No. 804002).

Data availability Some or all data, models or codes that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Karp RM (1972) Reducibility Among Combinatorial Problems. In: Miller RE, Thatcher JW (eds) Complexity of Computer Computations. Plenum, New York, pp 85–103
2. Chen J, Kanj IA, Jia W (2001) Vertex cover: further observations and further improvements. J Algorithms 41(2):280–301
3. Zhou Yupeng, Wang Yiyuan, Gao Jian, Luo Na, Wang Jianan (2018) An efficient local search for partial vertex cover problem. Neural Comput Appl 30:2245–2256
4. Bhattacharya S, Henzinger M, Italiano GF (2015) Deterministic fully dynamic data structures for vertex cover and matching. In: Proceedings of the Twenty-sixth annual ACM-SIAM symposium

on discrete algorithms, SODA '15, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, pp. 785–804
5. Safar M, Taha M, Habib S (2007) Modeling the communication problem in wireless sensor networks as a vertex cover. In: 2007 IEEE/ACS international conference on computer systems and applications, IEEE, pp. 592-598
6. Anderson JE, Chakrabortty A (2012) A minimum cover algorithm for pmu placement in power system networks under line observability constraints. In: 2012 IEEE power and energy society general meeting. IEEE:1–7
7. Li Ruizhi, Shuli Hu, Wang Yiyuan, Minghao Y (2017) A local search algorithm with tabu strategy and perturbation mechanism for generalized vertex cover problem. Neural Comput Appl 28:1775–1785
8. Yang Y, Li X (2013) Towards a snowdrift game optimization to vertex cover of networks. IEEE Trans Cybern 43(3):948–956
9. Wu J, Shen X, Jiao K (2019) Game-based memetic algorithm to the vertex cover of networks. IEEE Trans Cybern 49(3):974–988
10. Richter S, Helmert M, Gretton C (2007) A stochastic local search approach to vertex cover. KI 2007 Advances in artificial intelligence. Springer, Berlin, pp 412–426
11. Cai S, Su K, Chen Q (2010) Ewls: A new local search for minimum vertex cover. In: Twenty-Fourth AAAI conference on artificial intelligence, pp.45-50
12. Cai S, Su K, Sattar A (2011) Local search with edge weighting and configuration checking heuristics for minimum vertex cover. Artif Intell 175(9):1672–1696
13. Cai S, Su K, Luo C, Sattar A (2013) Numvc: an efficient local search algorithm for minimum vertex cover. J Artif Intell Res 46(1):687–716
14. Cai S, Lin J, Su K (2015) Two weighting local search for minimum vertex cover. In: Proceedings of the Twenty-Ninth AAAI conference on artificial intelligence, AAAI'15, AAAI Press, pp. 1107–1113
15. Cai S, Lin J, Luo C (2017) Finding a small vertex cover in massive sparse graphs: construct, local search, and preprocess. J Artif Intell Res 59:463–494
16. Li R, Hu S, Wang Y, Yin M (2017) A local search algorithm with tabu strategy and perturbation mechanism for generalized vertex cover problem. Neural Comput Appl 28(7):1775–1785
17. Quan Hangsheng, Guo Ping (2021) A local search method based on edge age strategy for minimum vertex cover problem in massive graphs. Exp Syst Appl 182:115185
18. Zhou Y, Wang Y, Gao J, Luo N, Wang J (2018) An efficient local search for partial vertex cover problem. Neural Comput Appl 30(7):2245–2256
19. Berge C (1984) Hypergraphs: combinatorics of finite sets. Elsevier, Amsterdam
20. Dinur I, Safra S (2005) On the hardness of approximating minimum vertex cover. Annals Math 163(1):439–485
21. Chen J, Kanj IA, Xia G (2010) Improved upper bounds for vertex cover. Theor Comput Sci 411:3736–3756
22. Akiba T, Iwata Y (2016) Branch-and-reduce exponential/FPT algorithms in practice: a case study of vertex cover. Theor Comput Sci 209:211–225
23. Füredi Z (1988) Matchings and covers in hypergraphs. Graphs Comb 4(1):115–206
24. Pawlak Z (1982) Rough sets. Int J Comput Inf Sci 11(5):341–356
25. Janicki R (2018) Approximations of arbitrary relations by partial orders. Int J Approx Reason 98:177–195
26. Xie X, Qin X (2018) Dynamic feature selection algorithm based on minimum vertex cover of hypergraph. Advances in knowledge discovery and data mining. Springer International Publishing, Cham, pp 40–51
27. Cattaneo G, Chiaselotti G, Ciucci D, Gentile T (2016) On the connection of hypergraph theory with formal concept analysis and rough set theory. Inf Sci 330:342–357
28. Chiaselotti G, Ciucci D, Gentile T, Infusino F (2016) Generalizations of rough set tools inspired by graph theory. Fundamenta Informaticae 148(1–2):207–227
29. Chen J, Lin Y, Lin G, Li J, Zhang Y (2017) Attribute reduction of covering decision systems by hypergraph model. Knowl-Based Syst 118:93–104
30. Chen J, Lin Y, Lin G, Li J, Ma Z (2015) The relationship between attribute reducts in rough sets and minimal vertex covers of graphs. Inf Sci 325:87–97
31. Skowron A, Rauszer C (1992) The discernibility matrices and functions in information systems. In: Słowiński R (ed) Intelligent decision support. Kluwer Academic Publishers, Dordrecht
32. Nguyen HS (2005) Approximate boolean reasoning approach to rough sets and data mining. In: Fuzzy Sets, Data Mining, Granular Computing (eds) Sets rough. Springer, Heidelberg, pp 12–22
33. Xie X, Qin X (2018) A novel incremental attribute reduction approach for dynamic incomplete decision systems. Int J Approx Reason 93:443–462
34. Huang Z, Li J (2021) A fitting model for attribute reduction with fuzzy β covering. Fuzzy Sets Syst 413:114–137
35. Abd El Aziz M, Hassanien AE (2018) An improved social spider optimization algorithm based on rough sets for solving minimum number attribute reduction problem. Neural Comput Appl 30(8):2441–2452
36. Chen J, Lin Y, Li J, Lin G, Ma Z, Tan A (2016) A rough set method for the minimum vertex cover problem of graphs. Appl Soft Comput 42:360–367
37. Zhou Q, Qin X, Xie X (2018) Dimension incremental feature selection approach for vertex cover of hypergraph using rough sets. IEEE Access 6:50142–50153
38. Xie X, Janicki R, Qin X, Zhao W, Huang G (2019) Local search for attribute reduction. Lecture notes in artificial intelligence. Springer, Berlin, pp 102–117
39. Zhuang S, Chen D (2019) A novel algorithm for the vertex cover problem based on minimal elements of discernibility matrix. Int J Machine Learn Cybern 10:3467–3474
40. Xie X, Qin X, Yu C, Xu X (2018) Test-cost-sensitive rough set based approach for minimum weight vertex cover problem. Appl Soft Comput 64(64):423–435
41. Rossi RA, Ahmed NK (2015) The network data repository with interactive graph analytics and visualization, http://networkrepository.com
42. Schlag S A (2017) Benchmark Set for Multilevel Hypergraph Partitioning Algorithms, https://doi.org/10.5281/zenodo.291466
43. Dheeru Dua, Casey Graff UCI Machine Learning Repository, http://www.ics.uci.edu/mlearn/MLRepository.html
44. Li J, Cheng K, Wang S, Morstatter F, Robert T, Tang J, Liu H (2017) Feature selection: a data perspective, arXiv:1601.07996